

OWASP Top 10

2025 Edition

Open Web Application Security Project

The Ten Most Critical Web Application Security Risks

Introduction

Welcome to the 8th installment of the OWASP Top Ten!

A huge thank you to everyone who contributed data and perspectives in the survey. Without you, this installment would not have been possible. **THANK YOU!**

Introducing the OWASP Top 10:2025

- A01:2025 - Broken Access Control
- A02:2025 - Security Misconfiguration
- A03:2025 - Software Supply Chain Failures
- A04:2025 - Cryptographic Failures
- A05:2025 - Injection
- A06:2025 - Insecure Design
- A07:2025 - Authentication Failures
- A08:2025 - Software or Data Integrity Failures
- A09:2025 - Security Logging & Alerting Failures
- A10:2025 - Mishandling of Exceptional Conditions

What's changed in the Top 10 for 2025

There are two new categories and one consolidation in the Top Ten for 2025. We've worked to maintain our focus on the root cause over the symptoms as much as possible. With the complexity of software engineering and software security, it's basically impossible to create ten categories without some level of overlap.

- **A01:2025 - Broken Access Control** maintains its position at #1 as the most serious application security risk; the contributed data indicates that on average, 3.73% of applications tested had one or more of the 40 Common Weakness Enumerations (CWEs) in this category. As indicated by the dashed line in the above figure, Server-Side Request Forgery (SSRF) has been rolled into this category.
- **A02:2025 - Security Misconfiguration** moved up from #5 in 2021 to #2 in 2025. Misconfigurations are more prevalent in the data for this cycle. 3.00% of the applications tested

had one or more of the 16 CWEs in this category. This is not surprising, as software engineering is continuing to increase the amount of an application's behavior that is based on configurations.

- **A03:2025 - Software Supply Chain Failures** is an expansion of A06:2021-Vulnerable and Outdated Components to include a broader scope of compromises occurring within or across the entire ecosystem of software dependencies, build systems, and distribution infrastructure. This category was overwhelmingly voted a top concern in the community survey. This category has 5 CWEs and a limited presence in the collected data, but we believe this is due to challenges in testing and hope that testing catches up in this area. This category has the fewest occurrences in the data, but also the highest average exploit and impact scores from CVEs.

- **A04:2025 - Cryptographic Failures** falls two spots from #2 to #4 in the ranking. The contributed data indicates that, on average, 3.80% of applications have one or more of the 32 CWEs in this category. This category often leads to sensitive data exposure or system compromise.

- **A05:2025 - Injection** falls two spots from #3 to #5 in the ranking, maintaining its position relative to Cryptographic Failures and Insecure Design. Injection is one of the most tested categories, with the greatest number of CVEs associated with the 38 CWEs in this category. Injection includes a range of issues from Cross-site Scripting (high frequency/low impact) to SQL Injection (low frequency/high impact) vulnerabilities.

- **A06:2025 - Insecure Design** slides two spots from #4 to #6 in the ranking as Security Misconfiguration and Software Supply Chain Failures leapfrog it. This category was introduced in 2021, and we have seen noticeable improvements in the industry related to threat modeling and a greater emphasis on secure design.

- **A07:2025 - Authentication Failures** maintains its position at #7 with a slight name change (previously it was "Identification and Authentication Failures") to more accurately reflect the 36 CWEs in this category. This category remains important, but the increased use of standardized frameworks for authentication appears to be having beneficial effects on the occurrences of authentication failures.

- **A08:2025 - Software or Data Integrity Failures** continues at #8 in the list. This category is focused on the failure to maintain trust boundaries and verify the integrity of software, code, and data artifacts at a lower level than Software Supply Chain Failures.

- **A09:2025 - Security Logging & Alerting Failures** retains its position at #9. This category has a slight name change (previously Security Logging and Monitoring Failures") to emphasize the importance of the alerting functionality needed to induce appropriate action on relevant logging events. Great logging with no alerting is of minimal value in identifying security incidents. This category will always be underrepresented in the data, and was again voted into a position in the list from the community survey participants.

- **A10:2025 - Mishandling of Exceptional Conditions** is a new category for 2025. This category contains 24 CWEs focusing on improper error handling, logical errors, failing open, and other related scenarios stemming from abnormal conditions that systems may encounter.

Methodology

This installment of the Top Ten remains data-informed, but not blindly data-driven. We ranked 12 categories based on the data contributed, and allowed two to be promoted or highlighted by responses from the community survey. We do this for a fundamental reason: examining the contributed data is essentially looking into the past. Application Security researchers dedicate time to identifying new vulnerabilities and developing new testing methods. It takes weeks to years to integrate these tests into tools and processes. By the time we can reliably test a weakness at scale, years may have passed. There are also important risks that we may never be able to reliably test and be present in the data. To balance that view, we use a community survey to ask application security and development practitioners on the front lines what they see as essential risks that may be underrepresented in the testing data.

How the categories are structured

A few categories have changed from the previous installment of the OWASP Top Ten. Here is a high-level summary of the category changes.

In this iteration, we asked for data, with no restriction on CWEs like we did for the 2021 edition. We asked for the number of applications tested for a given year (starting in 2021), and the number of applications with at least one instance of a CWE found in testing. This format allows us to track how prevalent each CWE is within the population of applications. We ignore frequency for our purposes; while it may be necessary for other situations, it only hides the actual prevalence in the application population. Whether an application has four instances of a CWE or 4,000 instances is not part of the calculation for the Top Ten. Especially as manual testers tend to only list a vulnerability once no matter how many times it is repeated in an application, while automated testing frameworks list every instance of a vulnerability as unique. We went from approximately 30 CWEs in 2017, to almost 400 CWEs in 2021, to 589 CWEs in this edition to analyze in the dataset. We plan to do additional data analysis as a supplement in the future. This significant increase in the number of CWEs necessitates changes to how the categories are structured.

We spent several months grouping and categorizing CWEs and could have continued for additional months. We had to stop at some point. There are both root cause and symptom types of CWEs, where root cause types are like "Cryptographic Failure" and "Misconfiguration" contrasted to symptom types like "Sensitive Data Exposure" and "Denial of Service." We decided to focus on the root cause whenever possible as it's more logical for providing identification and remediation guidance. Focusing on the root cause over the symptom isn't a new concept; the Top Ten has been a mix of symptom and root cause. CWEs are also a mix of symptom and root cause; we are simply being more deliberate about calling it out. There is an average of 25 CWEs per category in this installment, with the lower bounds at 5 CWEs for A03:2025-Software Supply Chain Failures and A09:2025 Security Logging and Alerting Failures to 40 CWEs in A01:2025-Broken Access Control. We made the decision to cap the number of CWEs in a category to 40. This updated category structure offers additional training benefits as companies can focus on CWEs that make sense for a language/framework.

We have been asked why not shift to a list of 10 CWEs as a Top 10, similar to the MITRE Top 25 Most Dangerous Software Weaknesses. There are two primary reasons why we use multiple CWEs in categories. First, not all CWEs exist in all programming languages or frameworks. This causes issues

for tooling and training/awareness programs as part of the Top Ten may not be applicable. The second reason is that there are multiple CWEs for common vulnerabilities. For example, there are multiple CWEs for general Injection, Command Injection, Cross-site Scripting, Hardcoded Passwords, Lack of Validation, Buffer Overflows, Cleartext Storage of Sensitive Information, and many others. Depending on the organization or tester, different CWEs might be used. By using a category with multiple CWEs we can help raise the baseline and awareness of the different types of weaknesses that may occur under a common category name. In this edition of the Top Ten 2025, there are 248 CWEs within the 10 categories. There are a total of 968 CWEs in the downloadable dictionary from MITRE at the time of this release.

How the data is used for selecting categories

Similar to what we did for the 2021 edition, we leveraged CVE data for *Exploitability* and *(Technical) Impact*. We downloaded OWASP Dependency Check and extracted the CVSS Exploit and Impact scores, grouping them by relevant CWEs listed with the CVEs. It took a fair bit of research and effort, as all the CVEs have CVSSv2 scores, but there are flaws in CVSSv2 that CVSSv3 should address. After a certain point in time, all CVEs are assigned a CVSSv3 score as well. Additionally, the scoring ranges and formulas were updated between CVSSv2 and CVSSv3.

In CVSSv2, both Exploit and (Technical) Impact could be up to 10.0, but the formula would knock them down to 60% for Exploit and 40% for Impact. In CVSSv3, the theoretical max was limited to 6.0 for Exploit and 4.0 for Impact. With the weighting considered, the Impact scoring shifted higher, almost a point and a half on average in CVSSv3, and exploitability moved nearly half a point lower on average.

There are approximately 175k records (up from 125k in 2021) of CVEs mapped to CWEs in the National Vulnerability Database (NVD), extracted from OWASP Dependency Check. Additionally, there are 643 unique CWEs mapped to CVEs (up from 241 in 2021). Within the nearly 220k CVEs that were extracted, 160k had CVSS v2 scores, 156k had CVSS v3 scores, and 6k had CVSS v4 scores. Many CVEs have multiple scores, which is why they total more than 220k.

For the Top Ten 2025, we calculated average exploit and impact scores in the following manner. We grouped all the CVEs with CVSS scores by CWE and weighted both exploit and impact scores by the percentage of the population that had CVSSv3, as well as the remaining population with CVSSv2 scores, to get an overall average. We mapped these averages to the CWEs in the dataset to use as Exploit and (Technical) Impact scoring for the other half of the risk equation.

Why not use CVSS v4.0, you may ask? That's because the scoring algorithm was fundamentally changed, and it no longer easily provides the *Exploit* or *Impact* scores as CVSS v2 and CVSSv3 do. We will attempt to figure out a way to use CVSS v4.0 scoring for future versions of the Top Ten, but we were unable to determine a timely way to do so for the 2025 edition.

Why we use a community survey

The results in the data are largely limited to what the industry can test for in an automated fashion. Talk to a seasoned AppSec professional, and they will tell you about stuff they find and trends they see that

aren't yet in the data. It takes time for people to develop testing methodologies for certain vulnerability types and then more time for those tests to be automated and run against a large population of applications. Everything we find is looking back in the past and might be missing trends from the last year, which are not present in the data.

Therefore, we only pick eight of the ten categories from the data because it's incomplete. The other two categories are from the Top 10 community survey. It allows the practitioners on the front lines to vote for what they see as the highest risks that might not be in the data (and may never be expressed in data).

Thank you to our data contributors

The following organizations (along with several anonymous donors) kindly donated data for over 2.8 millions applications to make this the largest and most comprehensive application security data set. Without you, this would not be possible.

- Accenture (Prague)
- Anonymous (multiple)
- Bugcrowd
- Contrast Security
- CryptoNet Labs
- Intuitor SoftTech Services
- Orca Security
- Probely
- Semgrep
- Sonar
- usd AG
- Veracode
- Wallarm

Lead Authors

- Andrew van der Stock - X: @vanderaj
- Brian Glas - X: @infosecdad
- Neil Smithline - X: @appsecneil
- Tanya Janca - X: @shehackspurple
- Torsten Gigler - Mastodon: @torsten_gigler@infosec.exchange

Log issues and pull requests

Please log any corrections or issues:

Project links:

- [Homepage](#)
- [GitHub repository](#)

About OWASP

The Open Worldwide Application Security Project (OWASP) is an open community dedicated to enabling organizations to develop, purchase, and maintain applications and APIs that can be trusted.

At OWASP, you'll find free and open:

- Application security tools and standards
- Cutting edge research
- Standard security controls and libraries
- Complete books on application security testing, secure code development, and secure code review
- Presentations and videos
- Cheat sheets on many common topics
- Chapters meetings held worldwide and online
- Events, training, and conferences.
- Google Groups

Learn more at: <https://owasp.org>.

All OWASP tools, documents, videos, presentations, and chapters are free and open to anyone interested in improving application security.

We advocate approaching application security as a people, process, and technology problem, because the most effective approaches to application security require improvements in all these areas.

OWASP is a different kind of organization. Our freedom from commercial pressures allows us to provide unbiased, practical, and cost-effective information about application security.

OWASP is not affiliated with any technology company, although we support the informed use of commercial security technology. OWASP produces many types of materials in a collaborative, transparent, and open way.

The OWASP Foundation is the non-profit entity that ensures the project's long-term success. Almost everyone associated with OWASP is a volunteer, including the OWASP board, chapter leaders, project leaders, and project members. We support innovative security research with grants and infrastructure.

Join us!

Copyright and License

Copyright © 2003-2025 The OWASP Foundation, Inc. This document is released under the Creative Commons Attribution Share-Alike 4.0 license. For any reuse or distribution, you must make it

clear to others the license terms of this work.

What are Application Security Risks?

Attackers can potentially use many different paths through your application to do harm to your business or organization. Each of these ways poses a potential risk that needs to be investigated.

<table>

<tr>

<td>

Threat Agents

</td>

<td>

Attack \

Vectors

</td>

<td>

Exploitability

</td>

<td>

Likelihood of Missing Security

<p style="text-align: center">

Controls

</td>

<td>

Technical

<p style="text-align: center">

Impacts

</td>

<td>

Business

<p style="text-align: center">

Impacts

</td>

By environment, \ dynamic by situation picture
By Application exposure (by environment
Avg Weighted Exploit
Missing Controls \ by average Incidence rate \
Weighed by coverage
Avg Weighted Impact
By Business

In our Risk Rating we have taken into account the universal parameters of exploitability, average likelihood of missing security controls for a weakness and its technical impacts.

Each organization is unique, and so are the threat actors for that organization, their goals, and the impact of any breach. If a public interest organization uses a content management system (CMS) for public information and a health system uses that same exact CMS for sensitive health records, the threat actors and business impacts can be very different for the same software. It is critical to understand the risk to your organization based on the exposure of the application, the applicable threat agents by situation picture (for targeted and undirected attacks by business and location) and the

individual business impacts.

How the data is used for selecting categories and ranking them

In 2017, we selected categories by incidence rate to determine likelihood, then ranked them by team discussion based on decades of experience for Exploitability, Detectability (also likelihood), and Technical Impact. For 2021, we used data for Exploitability and (Technical) Impact from the CVSSv2 and CVSSv3 scores in the National Vulnerability Database (NVD). For 2025, we continued the same methodology that we created in 2021.

We downloaded OWASP Dependency Check and extracted the CVSS Exploit, and Impact scores grouped by related CWEs. It took a fair bit of research and effort as all the CVEs have CVSSv2 scores, but there are flaws in CVSSv2 that CVSSv3 should address. After a certain point in time, all CVEs are assigned a CVSSv3 score as well. Additionally, the scoring ranges and formulas were updated between CVSSv2 and CVSSv3.

In CVSSv2, both Exploit and (Technical) Impact could be up to 10.0, but the formula would knock them down to 60% for Exploit and 40% for Impact. In CVSSv3, the theoretical max was limited to 6.0 for Exploit and 4.0 for Impact. With the weighting considered, the Impact scoring shifted higher, almost a point and a half on average in CVSSv3, and exploitability moved nearly half a point lower on average when we conducted analysis for the 2021 Top Ten.

There are approximately 175k records (up from 125k in 2021) of CVEs mapped to CWEs in the National Vulnerability Database (NVD), extracted from OWASP Dependency Check. Additionally, there are 643 unique CWEs mapped to CVEs (up from 241 in 2021). Within the nearly 220k CVEs that were extracted, 160k had CVSS v2 scores, 156k had CVSS v3 scores, and 6k had CVSS v4 scores. Many CVEs have multiple scores, which is why they total more than 220k.

For the Top Ten 2025, we calculated average exploit and impact scores in the following manner. We grouped all the CVEs with CVSS scores by CWE and weighted both exploit and impact scores by the percentage of the population that had CVSSv3, as well as the remaining population with CVSSv2 scores, to get an overall average. We mapped these averages to the CWEs in the dataset to use as Exploit and (Technical) Impact scoring for the other half of the risk equation.

Why not use CVSS v4.0, you may ask? That's because the scoring algorithm was fundamentally changed, and it no longer easily provides the *Exploit* or *Impact* scores as CVSSv2 and CVSSv3 do. We will attempt to figure out a way to use CVSS v4.0 scoring for future versions of the Top Ten, but we were unable to determine a timely way to do so for the 2025 edition.

For the incidence rate, we calculated the percentage of applications vulnerable to each CWE from the population tested by an org for a period of time. As a reminder, we are not using frequency (or how many times an issue appears in an application), we are interested in what percentage of the population of applications were found to have each CWE.

For coverage we look at the percentage of applications tested by all organizations for a given CWE. The higher the calculated coverage, the stronger the assurance that the incidence rate is accurate as

the sample size is more representative of the population.

The formula that we used for this iteration is similar to 2021, with some weighting changes:

$(\text{Max Incidence Rate \% } 1000) + (\text{Max Coverage \% } 100) + (\text{Avg Exploit } 10) + (\text{Avg Impact } 20) + (\text{Sum Occurrences} / 10000) = \text{Risk Score}$

The calculated scores ranged from 621.60 for the category of Broken Access Control to 271.08 for Memory Management Errors.

This is not a perfect system, but it is valuable for ranking risk categories.

One additional challenge that is growing is the definition of an "application". As the industry shifts to different architectures that consist of micro-services and other implementations that are smaller than a traditional application, calculations are more difficult. For instance, if an organization is testing code repositories, what does it consider an application? Similar to the growth of CVSSv4, the next edition of the Top Ten may need to adjust the analysis and scoring to account for a constantly changing industry.

Data Factors

There are data factors that are listed for each of the Top Ten Categories, here is what they mean:

CWEs Mapped: The number of CWEs mapped to a category by the Top Ten team.

Incidence Rate: Incidence rate is the percentage of applications vulnerable to that CWE from the population tested by that org for that year.

Weighted Exploit: The Exploit sub-score from CVSSv2 and CVSSv3 scores assigned to CVEs mapped to CWEs, normalized, and placed on a 10pt scale.

Weighted Impact: The Impact sub-score from CVSSv2 and CVSSv3 scores assigned to CVEs mapped to CWEs, normalized, and placed on a 10pt scale.

(Testing) Coverage: The percentage of applications tested by all organizations for a given CWE.

Total Occurrences: Total number of applications found to have the CWEs mapped to a category.

Total CVEs: Total number of CVEs in the NVD DB that were mapped to the CWEs mapped to a category.

Formula: $(\text{Max Incidence Rate \% } 1000) + (\text{Max Coverage \% } 100) + (\text{Avg Exploit } 10) + (\text{Avg Impact } 20) + (\text{Sum Occurrences} / 10000) = \text{Risk Score}$

Establishing a Modern Application Security Program

The OWASP Top Ten lists are awareness documents, meant to bring awareness to the most critical risks of whichever topic they cover. They are not meant to be a complete list, only a starting place. In previous versions of this list we have prescribed starting an application security program as the best way to avoid these risks, and more. In this section we will cover how to start and build a modern application security program.

If you already have an application security program, consider performing a maturity assessment on it using OWASP SAMM (Software Assurance Maturity Model) or DSOMM (DevSecOps Maturity Model) . These maturity models are comprehensive and exhaustive and can be used to help you figure out where you should best focus your efforts for expanding and maturing your program. Please note: you do not need to do everything in OWASP SAMM or DSOMM to be doing a good job, they are meant to guide you and offer many options. They are not meant to offer unattainable standards or describe unaffordable programs. They are expansive in order to offer you many ideas and options.

If you are starting a program from scratch, or you find OWASP SAMM or DSOMM 'too much' for your team right now, please review the following advice.

1. Establish a Risk Based Portfolio Approach:

- Identify the protection needs of your application portfolio from a business perspective. This should be driven in part by privacy laws and other regulations relevant to the data asset being protected.
- Establish a common risk rating model with a consistent set of likelihood and impact factors reflective of your organization's tolerance for risk.
- Accordingly measure and prioritize all your applications and APIs. Add the results to your Configuration Management Database (CMDB).
- Establish assurance guidelines to properly define coverage and level of rigor required.

2. Enable with a Strong Foundation:

- Establish a set of focused policies and standards that provide an application security baseline for all development teams to adhere to.
- Define a common set of reusable security controls that complement these policies and standards and provide design and development guidance on their use.
- Establish an application security training curriculum that is required and targeted to different development roles and topics.

3. Integrate Security into Existing Processes:

- Define and integrate secure implementation and verification activities into existing development and operational processes.
- Activities include threat modeling, secure design and design review, secure coding and code review, penetration testing, and remediation.
- Provide subject matter experts and support services for development and project teams to be successful.
- Review your current system development life cycle and all software security activities, tooling, policies, and processes, then document them.
- For new software, add one or more security activities to each phase of the system development life cycle (SDLC). Below we offer many suggestions of what you can do below. Ensure you perform these new activities on every new project or software initiative, this way you will know each new piece of software will be delivered at an acceptable security posture for your organizations.
- Select your activities to ensure your final product meets an acceptable level of risk for your organization.
- For existing software (sometimes called legacy) you will want to have a formal maintenance plan, please look below for ideas of how to maintain secure applications in the section called 'Operations and Change Management'.

4. Application Security Education:

- Consider starting a security champion program, or general security education program for your developers (sometimes called an advocacy or security awareness program), to teach them everything you wish they would know. This will keep them up to date, help them know how to do their work securely, and make the security culture where you work more positive. It often also improves trust between the teams and makes for a happier working relationship. OWASP supports you in this with the OWASP Security Champions Guide, which is being expanded step by step.
- The OWASP Education Project provides training materials to help educate developers on web application security. For hands-on learning about vulnerabilities, try the OWASP Juice Shop Project, or OWASP WebGoat. To stay current, come to an OWASP AppSec Conference, OWASP Conference Training, or local OWASP Chapter meetings.

5. Provide Management Visibility:

- Manage with metrics. Drive improvement and funding decisions based on the metrics and analysis data captured. Metrics include adherence to security practices and activities, vulnerabilities introduced, vulnerabilities mitigated, application coverage, defect density by type and instance counts, etc.
- Analyze data from the implementation and verification activities to look for root cause and vulnerability patterns to drive strategic and systemic improvements across the enterprise. Learn

from mistakes and offer positive incentives to promote improvements.

Establish & Use Repeatable Security Processes and Standard Security Controls

Requirements and Resource Management Phase:

- Collect and negotiate the business requirements for an application with the business, including the protection requirements with regard to confidentiality, authenticity, integrity and availability of all data assets, and the expected business logic.
- Compile the technical requirements including functional and nonfunctional security requirements. OWASP recommends you use the [OWASP Application Security Verification Standard (ASVS)](<https://owasp.org/www-project-application-security-verification-standard/>) as a guide for setting the security requirements for your application(s).
- Plan and negotiate the budget that covers all aspects of design, build, testing and operation, including security activities.
- Add security activities to your project schedule.
- Introduce yourself as the security representative at the project kick off, so they know who to talk to.

Request for Proposals (RFP) and Contracting:

- Negotiate the requirements with internal or external developers, including guidelines and security requirements with respect to your security program, e.g. SDLC, best practices.
- Rate the fulfillment of all technical requirements, including a planning and design phase.
- Negotiate all technical requirements, including design, security, and service level agreements (SLA).
- Adopt templates and checklists, such as OWASP Secure Software Contract Annex.
Note:
The annex is for US contract law, so please consult qualified legal advice before using the sample annex.

Planning and Design Phase:

- Negotiate planning and design with the developers and internal shareholders, e.g. security specialists.
- Define the security architecture, controls, countermeasures and design reviews appropriate to the protection needs and the expected threat level. This should be supported by security specialists.
- Rather than retrofitting security into your applications and APIs, it is far more cost effective to design the security in from the start. OWASP recommends the OWASP Cheat Sheets and the

OWASP Proactive Controls as a good starting point for guidance on how to design security included from the beginning.

- Perform threat modelling, see OWASP Cheat Sheet: Threat Modeling.
- Teach your software architects secure design concepts and patterns and ask them to add them to their designs where possible.
- Examine data flows with your developers.
- Add security user stories alongside all of your other user stories.

Secure Development Lifecycle:

- To improve the process your organization follows when building applications and APIs, OWASP recommends the OWASP Software Assurance Maturity Model (SAMM). This model helps organizations formulate and implement a strategy for software security that is tailored to the specific risks facing their organization.
- Provide secure coding training to your software developers, and any other training you think will help them create more robust and secure applications.
- Code review, see OWASP Cheat Sheet: Secure Code Review.
- Give your developers security tools, then teach them how to use them, especially static analysis, software composition analysis, secret, and Infrastructure-as-Code (IaC) scanners.
- Create guardrails for your developers, if possible (technical safeguards to steer them towards more secure choices).
- Building strong and usable security controls is difficult. Offer secure defaults whenever possible, and create 'paved roads' (making the easiest way also the most secure way to do something, the obvious preferred way) whenever possible. The OWASP Cheat Sheets are a good starting point for developers, and many modern frameworks now come with standard and effective security controls for authorization, validation, CSRF prevention, etc.
- Give your developers security-related IDE plugins and encourage them to use them.
- Provide them a secret management tool, licenses, and documentation on how to use it.
- Provide them a private AI to use, ideally set up with a RAG server full of useful security documentation, prompts your team has written for better outcomes, and an MCP server that calls the security tooling of choice for your org. Teach them how to use AI safely, because they are going to do it whether you like it or not.

Establish Continuous Application Security Testing:

- Test the technical functions and integration with the IT architecture and coordinate business tests.
- Create "use" and "abuse" test cases from technical and business perspectives.

- Manage security tests according to internal processes, the protection needs, and the assumed threat level by the application.
- Provide security testing tools (fuzzers, DAST, etc.), a safe place to test, and training on how to use them, OR do the testing for them OR hire a tester
- If you require a high level of assurance, consider a formal penetration test, as well as stress testing and performance testing.
- Work with your developers to help them decide what they need to fix from the bug reports, and ensure their managers give them time to do it.

Rollout:

- Put the application in operation and migrate from previously used applications if needed.
- Finalize all documentation, including the change management database (CMDB) and security architecture.

Operations and Change Management:

- Operations must include guidelines for the security management of the application (e.g. patch management).
- Raise the security awareness of users and manage conflicts about usability vs. security.
- Plan and manage changes, e.g. migrate to new versions of the application or other components like OS, middleware, and libraries.
- Ensure all apps are in your inventory, with all important details documented. Update all documentation, including in the CMDB and the security architecture, controls, and countermeasures, including any runbooks or project documentation.
- Perform logging, monitoring, and alerting for all apps. Add it if it's missing.
- Create processes for effective and efficient updating and patching.
- Create regular scanning schedules (hopefully dynamic, static, secrets, IaC, and software composition analysis).
- SLAs for fixing security bugs.
- Provide a way for employees (and ideally also your customers) to report bugs.
- Establish a trained incident response team that understands what software attacks look like, observability tooling.
- Run blocking or shielding tools to stop automated attacks.
- Annual (or more often) hardening of configurations.
- At least annual penetration testing (depending upon the level assurance required for your app).

- Establish processes and tooling for hardening and protecting your software supply chain.
- Establish and update business continuity and disaster recovery planning that includes your most important applications and the tools you use to maintain them.

Retiring Systems:

- Any required data should be archived. All other data should be securely wiped.
- Securely retire the application, including deleting unused accounts and roles and permissions.
- Set your application's state to retired in the CMDB.

Using the OWASP Top 10 as a standard

The OWASP Top 10 is primarily an awareness document. However, this has not stopped organizations from using it as a de facto industry AppSec standard since its inception in 2003. If you want to use the OWASP Top 10 as a coding or testing standard, know that it is the bare minimum and just a starting point.

One of the difficulties of using the OWASP Top 10 as a standard is that we document AppSec risks, and not necessarily easily testable issues. For example, A06:2025-Insecure Design is beyond the scope of most forms of testing. Another example is testing whether in-place, in-use, and effective logging and monitoring are implemented, which can only be done with interviews and requesting a sampling of effective incident responses. A static code analysis tool can look for the absence of logging, but it might be impossible to determine if business logic or access control is logging critical security breaches. Penetration testers may only be able to determine that they have invoked incident response in a test environment, which is rarely monitored in the same way as production.

Here are our recommendations for when it is appropriate to use the OWASP Top 10:

Use Case	OWASP Top 10 2025	OWASP Application Security Verification Standard
		Awareness

Yes
Training
Entry level
Comprehensive
Design and architecture
Occasionally
Yes
Coding standard
Bare minimum
Yes
Secure Code review

</td>

<td>Bare minimum

</td>

<td>Yes

</td>

</tr>

<tr>

<td>Peer review checklist

</td>

<td>Bare minimum

</td>

<td>Yes

</td>

</tr>

<tr>

<td>Unit testing

</td>

<td>Occasionally

</td>

<td>Yes

</td>

</tr>

<tr>

<td>Integration testing

</td>

<td>Occasionally

</td>

<td>Yes

</td>

</tr>

<tr>

<td>Penetration testing

	Bare minimum
	Yes
	Tool support
	Bare minimum
	Yes
	Secure Supply Chain
	Occasionally
	Yes

We would encourage anyone wanting to adopt an application security standard to use the OWASP Application Security Verification Standard (ASVS), as it's designed to be verifiable and tested, and can be used in all parts of a secure development lifecycle.

The ASVS is the only acceptable choice for tool vendors. Tools cannot comprehensively detect, test, or protect against the OWASP Top 10 due to the nature of several of the OWASP Top 10 risks, with reference to A06:2025-Insecure Design. OWASP discourages any claims of full coverage of the OWASP Top 10, because it's simply untrue.

A01:2025 Broken Access Control

Background.

Maintaining its position at #1 in the Top Ten, 100% of the applications tested were found to have some form of broken access control. Notable CWEs included are *CWE-200: Exposure of Sensitive Information to an Unauthorized Actor*, *CWE-201: Exposure of Sensitive Information Through Sent Data*, *CWE-918 Server-Side Request Forgery (SSRF)*, and *CWE-352: Cross-Site Request Forgery (CSRF)*. This category has the highest number of occurrences in the contributed data, and second highest number of related CVEs.

Score table.

CWEs Mapped
Max Incidence Rate
Avg Incidence Rate
Max Coverage
Avg Coverage
Avg Weighted Exploit
Avg Weighted Impact
Total Occurrences
Total CVEs

40	
20.15%	
3.74%	
100.00%	
42.93%	
7.04	
3.84	
1,839,701	
32,654	

Description.

Access control enforces policy such that users cannot act outside of their intended permissions. Failures typically lead to unauthorized information disclosure, modification or destruction of all data, or performing a business function outside the user's limits. Common access control vulnerabilities include:

- Violation of the principle of least privilege, commonly known as deny by default, where access should only be granted for particular capabilities, roles, or users, but is available to anyone.
- Bypassing access control checks by modifying the URL (parameter tampering or force browsing), internal application state, or the HTML page, or by using an attack tool that modifies API requests.
- Permitting viewing or editing someone else's account by providing its unique identifier (insecure direct object references)

- An accessible API with missing access controls for POST, PUT, and DELETE.
- Elevation of privilege. Acting as a user without being logged in or or gaining privileges beyond those expected of the logged in user (e.g. admin access).
- Metadata manipulation, such as replaying or tampering with a JSON Web Token (JWT) access control token, a cookie or hidden field manipulated to elevate privileges, or abusing JWT invalidation.
- CORS misconfiguration allows API access from unauthorized or untrusted origins.
- Force browsing (guessing URLs) to authenticated pages as an unauthenticated user or to privileged pages as a standard user.

How to prevent.

Access control is only effective when implemented in trusted server-side code or serverless APIs, where the attacker cannot modify the access control check or metadata.

- Except for public resources, deny by default.
- Implement access control mechanisms once and reuse them throughout the application, including minimizing Cross-Origin Resource Sharing (CORS) usage.
- Model access controls should enforce record ownership rather than allowing users to create, read, update, or delete any record.
- Unique application business limit requirements should be enforced by domain models.
- Disable web server directory listing and ensure file metadata (e.g., .git) and backup files are not present within web roots.
- Log access control failures, alert admins when appropriate (e.g., repeated failures).
- Implement rate limits on API and controller access to minimize the harm from automated attack tooling.
- Stateful session identifiers should be invalidated on the server after logout. Stateless JWT tokens should be short-lived to minimize the window of opportunity for an attacker. For longer-lived JWTs, consider using refresh tokens and following OAuth standards to revoke access.
- Use well-established toolkits or patterns that provide simple, declarative access controls.

Developers and QA staff should include functional access control in their unit and integration tests.

Example attack scenarios.

Scenario #1: The application uses unverified data in an SQL call that is accessing account information:

```
pstmt.setString(1, request.getParameter("acct"));
ResultSet results = pstmt.executeQuery( );
```

An attacker can simply modify the browser's 'acct' parameter to send any desired account number. If not correctly verified, the attacker can access any user's account.

```
https://example.com/app/accountInfo?acct=notmyacct
```

Scenario #2: An attacker simply forces browsers to target URLs. Admin rights are required for access to the admin page.

```
https://example.com/app/getappInfo  
https://example.com/app/admin_getappInfo
```

If an unauthenticated user can access either page, it's a flaw. If a non-admin can access the admin page, this is a flaw.

Scenario #3: An application puts all of their access control in their front-end. While the attacker cannot get to `https://example.com/app/admin_getappInfo` due to JavaScript code running in the browser, they can simply execute:

```
$ curl https://example.com/app/admin_getappInfo
```

from the command line.

References.

- OWASP Proactive Controls: C1: Implement Access Control
- OWASP Application Security Verification Standard: V8 Authorization
- OWASP Testing Guide: Authorization Testing
- OWASP Cheat Sheet: Authorization
- PortSwigger: Exploiting CORS misconfiguration
- OAuth: Revoking Access

List of Mapped CWEs

- CWE-22 Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')
- CWE-23 Relative Path Traversal
- CWE-36 Absolute Path Traversal
- CWE-59 Improper Link Resolution Before File Access ('Link Following')
- CWE-61 UNIX Symbolic Link (Symlink) Following
- CWE-65 Windows Hard Link
- CWE-200 Exposure of Sensitive Information to an Unauthorized Actor
- CWE-201 Exposure of Sensitive Information Through Sent Data
- CWE-219 Storage of File with Sensitive Data Under Web Root

- CWE-276 Incorrect Default Permissions
- CWE-281 Improper Preservation of Permissions
- CWE-282 Improper Ownership Management
- CWE-283 Unverified Ownership
- CWE-284 Improper Access Control
- CWE-285 Improper Authorization
- CWE-352 Cross-Site Request Forgery (CSRF)
- CWE-359 Exposure of Private Personal Information to an Unauthorized Actor
- CWE-377 Insecure Temporary File
- CWE-379 Creation of Temporary File in Directory with Insecure Permissions
- CWE-402 Transmission of Private Resources into a New Sphere ('Resource Leak')
- CWE-424 Improper Protection of Alternate Path
- CWE-425 Direct Request ('Forced Browsing')
- CWE-441 Unintended Proxy or Intermediary ('Confused Deputy')
- CWE-497 Exposure of Sensitive System Information to an Unauthorized Control Sphere
- CWE-538 Insertion of Sensitive Information into Externally-Accessible File or Directory
- CWE-540 Inclusion of Sensitive Information in Source Code
- CWE-548 Exposure of Information Through Directory Listing
- CWE-552 Files or Directories Accessible to External Parties
- CWE-566 Authorization Bypass Through User-Controlled SQL Primary Key
- CWE-601 URL Redirection to Untrusted Site ('Open Redirect')
- CWE-615 Inclusion of Sensitive Information in Source Code Comments
- CWE-639 Authorization Bypass Through User-Controlled Key
- CWE-668 Exposure of Resource to Wrong Sphere
- CWE-732 Incorrect Permission Assignment for Critical Resource
- CWE-749 Exposed Dangerous Method or Function
- CWE-862 Missing Authorization
- CWE-863 Incorrect Authorization
- CWE-918 Server-Side Request Forgery (SSRF)

- CWE-922 Insecure Storage of Sensitive Information
- CWE-1275 Sensitive Cookie with Improper SameSite Attribute

A02:2025 Security Misconfiguration

Background.

Moving up from #5 in the previous edition, 100% of the applications tested were found to have some form of misconfiguration, with an average incidence rate of 3.00%, and over 719k occurrences of a Common Weakness Enumeration (CWE) in this risk category. With more shifts into highly configurable software, it's not surprising to see this category moving up. Notable CWEs included are *CWE-16 Configuration* and *CWE-611 Improper Restriction of XML External Entity Reference (XXE)*.

Score table.

CWEs Mapped
Max Incidence Rate
Avg Incidence Rate
Max Coverage
Avg Coverage
Avg Weighted Exploit
Avg Weighted Impact
Total Occurrences
Total CVEs

16	
27.70%	
3.00%	
100.00%	
52.35%	
7.96	
3.97	
719,084	
1,375	

Description.

Security misconfiguration is when a system, application, or cloud service is set up incorrectly from a security perspective, creating vulnerabilities.

The application might be vulnerable if:

- It is missing appropriate security hardening across any part of the application stack or improperly configured permissions on cloud services.
- Unnecessary features are enabled or installed (e.g., unnecessary ports, services, pages, accounts, testing frameworks, or privileges).
- Default accounts and their passwords are still enabled and unchanged.

- A lack of central configuration for intercepting excessive error messages. Error handling reveals stack traces or other overly informative error messages to users.
- For upgraded systems, the latest security features are disabled or not configured securely.
- Excessive prioritization of backward compatibility leading to insecure configuration.
- The security settings in the application servers, application frameworks (e.g., Struts, Spring, ASP.NET), libraries, databases, etc., are not set to secure values.
- The server does not send security headers or directives, or they are not set to secure values.

Without a concerted, repeatable application security configuration hardening process, systems are at a higher risk.

How to prevent.

Secure installation processes should be implemented, including:

- A repeatable hardening process enabling the fast and easy deployment of another environment that is appropriately locked down. Development, QA, and production environments should all be configured identically, with different credentials used in each environment. This process should be automated to minimize the effort required to set up a new secure environment.
- A minimal platform without any unnecessary features, components, documentation, or samples. Remove or do not install unused features and frameworks.
- A task to review and update the configurations appropriate to all security notes, updates, and patches as part of the patch management process (see A03 Software Supply Chain Failures). Review cloud storage permissions (e.g., S3 bucket permissions).
- A segmented application architecture provides effective and secure separation between components or tenants, with segmentation, containerization, or cloud security groups (ACLs).
- Sending security directives to clients, e.g., Security Headers.
- An automated process to verify the effectiveness of the configurations and settings in all environments.
- Proactively add a central configuration to intercept excessive error messages as a backup.
- If these verifications are not automated, they should be manually verified annually at a minimum.
- Use identity federation, short-lived credentials, or role-based access mechanisms provided by the underlying platform instead of embedding static keys or secrets in code, configuration files, or pipelines.

Example attack scenarios.

Scenario #1: The application server comes with sample applications not removed from the production server. These sample applications have known security flaws that attackers use to compromise the server. Suppose one of these applications is the admin console, and default accounts weren't changed. In that case, the attacker logs in with the default password and takes over.

Scenario #2: Directory listing is not disabled on the server. An attacker discovers they can simply list directories. The attacker finds and downloads the compiled Java classes, which they decompile and reverse engineer to view the code. The attacker then finds a severe access control flaw in the application.

Scenario #3: The application server's configuration allows detailed error messages, such as stack traces to be returned to users. This potentially exposes sensitive information or underlying flaws, such as component versions that are known to be vulnerable.

Scenario #4: A cloud service provider (CSP) defaults to having sharing permissions open to the Internet. This allows sensitive data stored within cloud storage to be accessed.

References.

- OWASP Testing Guide: Configuration Management
- OWASP Testing Guide: Testing for Error Codes
- Application Security Verification Standard V13 Configuration
- NIST Guide to General Server Hardening
- CIS Security Configuration Guides/Benchmarks
- Amazon S3 Bucket Discovery and Enumeration
- ScienceDirect: Security Misconfiguration

List of Mapped CWEs

- CWE-5 J2EE Misconfiguration: Data Transmission Without Encryption
- CWE-11 ASP.NET Misconfiguration: Creating Debug Binary
- CWE-13 ASP.NET Misconfiguration: Password in Configuration File
- CWE-15 External Control of System or Configuration Setting
- CWE-16 Configuration
- CWE-260 Password in Configuration File
- CWE-315 Cleartext Storage of Sensitive Information in a Cookie
- CWE-489 Active Debug Code
- CWE-526 Exposure of Sensitive Information Through Environmental Variables
- CWE-547 Use of Hard-coded, Security-relevant Constants
- CWE-611 Improper Restriction of XML External Entity Reference
- CWE-614 Sensitive Cookie in HTTPS Session Without 'Secure' Attribute

- CWE-776 Improper Restriction of Recursive Entity References in DTDs ('XML Entity Expansion')
- CWE-942 Permissive Cross-domain Policy with Untrusted Domains
- CWE-1004 Sensitive Cookie Without 'HttpOnly' Flag
- CWE-1174 ASP.NET Misconfiguration: Improper Model Validation

A03:2025 Software Supply Chain Failures

Background.

This was top-ranked in the Top 10 community survey with exactly 50% respondents ranking it #1. Since initially appearing in the 2013 Top 10 as "A9 – Using Components with Known Vulnerabilities", the risk has grown in scope to include all supply chain failures, not just ones involving known vulnerabilities. Despite this increased scope, supply chain failures continue to be a challenge to identify with only 11 Common Vulnerability and Exposures (CVEs) having the related CWEs. However, when tested and reported in the contributed data, this category has the highest average incidence rate at 5.19%. The relevant CWEs are *CWE-477: Use of Obsolete Function*, *CWE-1104: Use of Unmaintained Third Party Components*, *CWE-1329: Reliance on Component That is Not Updateable*, and *CWE-1395: Dependency on Vulnerable Third-Party Component*.

Score table.

CWEs Mapped
Max Incidence Rate
Avg Incidence Rate
Max Coverage
Avg Coverage
Avg Weighted Exploit
Avg Weighted Impact
Total Occurrences

<td>Total CVEs

</td>

</tr>

<tr>

<td>6

</td>

<td>9.56%

</td>

<td>5.72%

</td>

<td>65.42%

</td>

<td>27.47%

</td>

<td>8.17

</td>

<td>5.23

</td>

<td>215,248

</td>

<td>11

</td>

</tr>

</table>

Description.

Software supply chain failures are breakdowns or other compromises in the process of building, distributing, or updating software. They are often caused by vulnerabilities or malicious changes in third-party code, tools, or other dependencies that the system relies on.

You are likely vulnerable if:

- you do not carefully track the versions of all components that you use (both client-side and server-side). This includes components you directly use as well as nested (transitive)

dependencies.

- the software is vulnerable, unsupported, or out of date. This includes the OS, web/application server, database management system (DBMS), applications, APIs and all components, runtime environments, and libraries.
- you do not scan for vulnerabilities regularly and subscribe to security bulletins related to the components you use.
- you do not have a change management process or tracking of changes within your supply chain, including tracking IDEs, IDE extensions and updates, changes to your organization's code repository, sandboxes, image and library repositories, the way artifacts are created and stored, etc. Every part of your supply chain should be documented, especially changes.
- you have not hardened every part of your supply chain, with a special focus on access control and the application of least privilege.
- your supply chain systems do not have any separation of duty. No single person should be able to write code and promote it all the way to production without oversight from another human being.
- components from untrusted sources, across any part of the tech stack, are used in or can impact on production environments.
- you do not fix or upgrade the underlying platform, frameworks, and dependencies in a risk-based, timely fashion. This commonly happens in environments when patching is a monthly or quarterly task under change control, leaving organizations open to days or months of unnecessary exposure before fixing vulnerabilities.
- software developers do not test the compatibility of updated, upgraded, or patched libraries.
- you do not secure the configurations of every part of your system (see A02:2025-Security Misconfiguration).
- your CI/CD pipeline has weaker security than the systems it builds and deploys, especially if it is complex.

How to prevent.

There should be a patch management process in place to:

- Centrally generate and manage the Software Bill of Materials (SBOM) of your entire software.
- Track not just your direct dependencies, but their (transitive) dependencies, and so on.
- Reduce attack surface by removing unused dependencies, unnecessary features, components, files, and documentation.
- Continuously inventory the versions of both client-side and server-side components (e.g., frameworks, libraries) and their dependencies using tools like OWASP Dependency Track, OWASP Dependency Check, retire.js, etc.
- Continuously monitor sources like Common Vulnerability and Exposures (CVE), National Vulnerability Database (NVD), and Open Source Vulnerabilities (OSV) for vulnerabilities in the components you use. Use software composition analysis, software supply chain, or security-focused SBOM tools to automate the process. Subscribe to alerts for security

vulnerabilities related to components you use.

- Only obtain components from official (trusted) sources over secure links. Prefer signed packages to reduce the chance of including a modified, malicious component (see A08:2025-Software and Data Integrity Failures).
- Deliberately choose which version of a dependency you use and upgrade only when there is need.
- Monitor for libraries and components that are unmaintained or do not create security patches for older versions. If patching is not possible, consider migrating to an alternative. If that is not possible, consider deploying a virtual patch to monitor, detect, or protect against the discovered issue.
- Update your CI/CD, IDE, and any other developer tooling regularly
- Avoid deploying updates to all systems simultaneously. Use staged rollouts or canary deployments to limit exposure in case a trusted vendor is compromised.

There should be a change management process or tracking system in place to track changes to:

- CI/CD settings (all build tools and pipelines)
- Code repositories
- Sandbox areas
- Developer IDEs
- SBOM tooling, and created artifacts
- Logging systems and logs
- Third party integrations, such as SaaS
- Artifact repositories
- Container registries

Harden the following systems, which includes enabling MFA and locking down IAM:

- Your code repository (which includes not checking in secrets, protecting branches, backups)
- Developer workstations (regular patching, MFA, monitoring, and more)
- Your build server & CI/CD (separation of duties, access control, signed builds, environment-scoped secrets, tamper-evident logs, more)
- Your artifacts (ensure integrity via provenance, signing, and time stamping, promote artifacts rather than rebuilding for each environment, ensure builds are immutable)
- Infrastructure as code (managed like all code, including use of PRs and version control)

Every organization must ensure an ongoing plan for monitoring, triaging, and applying updates or configuration changes for the lifetime of the application or portfolio.

Example attack scenarios.

Scenario #1: A trusted vendor is compromised with malware, leading to your computer systems being compromised when you upgrade. The most famous example of this is probably:

- The 2019 SolarWinds compromise that led to ~18,000 organizations being compromised. <https://www.npr.org/2021/04/16/985439655/a-worst-nightmare-cyberattack-the-untold-story-of-the-solarwinds-hack>

Scenario #2: A trusted vendor is compromised such that it behaves maliciously only under a specific condition.

- The 2025 Bybit theft of \$1.5 billion was caused by a supply chain attack in wallet software that only executed when the target wallet was being used.

Scenario #3: The Shai-HuLud supply chain attack in 2025 was the first successful self-propagating npm worm. Attacks seeded malicious versions of popular packages, which used a post-install script to harvest and exfiltrate sensitive data to public GitHub repositories. The malware would also detect npm tokens in the victim environment, and automatically use them to push malicious versions of any accessible package. The worm reached over 500 package versions before being disrupted by npm. This supply chain attack was advanced, fast-spreading, and damaging, and by targeting developer machines it demonstrated developers themselves are now prime targets for supply chain attacks.

Scenario #4: Components typically run with the same privileges as the application itself, so flaws in any component can result in serious impact. Such flaws can be accidental (e.g., coding error) or intentional (e.g., a backdoor in a component). Some example exploitable component vulnerabilities discovered are:

- CVE-2017-5638, a Struts 2 remote code execution vulnerability that enables the execution of arbitrary code on the server, has been blamed for significant breaches.
- CVE-2021-44228 ("Log4Shell"), an Apache Log4j remote code execution zero-day vulnerability, has been blamed for ransomware, cryptomining, and other attack campaigns.

References

- OWASP Application Security Verification Standard: V15 Secure Coding and Architecture
- OWASP Cheat Sheet Series: Dependency Graph SBOM
- OWASP Cheat Sheet Series: Vulnerable Dependency Management
- OWASP Dependency-Track
- OWASP CycloneDX
- OWASP Application Security Verification Standard: V1 Architecture, design and threat modelling
- OWASP Dependency Check (for Java and .NET libraries)
- OWASP Testing Guide - Map Application Architecture (OTG-INFO-010)
- OWASP Virtual Patching Best Practices
- The Unfortunate Reality of Insecure Libraries

- MITRE Common Vulnerabilities and Exposures (CVE) search
- National Vulnerability Database (NVD)
- Retire.js for detecting known vulnerable JavaScript libraries
- GitHub Advisory Database
- Ruby Libraries Security Advisory Database and Tools
- SAFECODE Software Integrity Controls (PDF)
- Glassworm supply chain attack
- PhantomRaven supply chain attack campaign

List of Mapped CWEs

- CWE-447 Use of Obsolete Function
- CWE-1035 2017 Top 10 A9: Using Components with Known Vulnerabilities
- CWE-1104 Use of Unmaintained Third Party Components
- CWE-1329 Reliance on Component That is Not Updateable
- CWE-1357 Reliance on Insufficiently Trustworthy Component
- CWE-1395 Dependency on Vulnerable Third-Party Component

A04:2025 Cryptographic Failures

Background.

Moving down two positions to #4, this weakness focuses on failures related to the lack of cryptography, insufficiently strong cryptography, leaking of cryptographic keys, and related errors. Three of the most common Common Weakness Enumerations (CWEs) in this risk involved the use of a weak pseudo-random number generator: *CWE-327 Use of a Broken or Risky Cryptographic Algorithm*, *CWE-331: Insufficient Entropy*, *CWE-1241: Use of Predictable Algorithm in Random Number Generator*, and *CWE-338 Use of Cryptographically Weak Pseudo-Random Number Generator (PRNG)*.

Score table.

CWEs Mapped
Max Incidence Rate
Avg Incidence Rate
Max Coverage
Avg Coverage
Avg Weighted Exploit
Avg Weighted Impact
Total Occurrences
Total CVEs

32	
13.77%	
3.80%	
100.00%	
47.74%	
7.23	
3.90	
1,665,348	
2,185	

Description.

Generally speaking, all data in transit should be encrypted at the transport layer (OSI layer 4). Previous hurdles such as CPU performance and private key/certificate management are now handled by CPUs having instructions designed to accelerate encryption (eg: AES support) and private key and certificate management being simplified by services like LetsEncrypt.org with major cloud vendors providing even more tightly integrated certificate management services for their specific platforms.

Beyond securing the transport layer, it is important to determine what data needs encryption at rest as well as what data needs extra encryption in transit (at the application layer, OSI layer 7). For example, passwords, credit card numbers, health records, personal information, and business secrets require

extra protection, especially if that data falls under privacy laws, e.g., EU's General Data Protection Regulation (GDPR), or regulations such as PCI Data Security Standard (PCI DSS). For all such data:

- Are any old or weak cryptographic algorithms or protocols used either by default or in older code?
- Are default crypto keys in use, are weak crypto keys generated, are keys re-used, or is proper key management and rotation missing?
- Are crypto keys checked into source code repositories?
- Is encryption not enforced, e.g., are any HTTP headers (browser) security directives or headers missing?
- Is the received server certificate and the trust chain properly validated?
- Are initialization vectors ignored, reused, or not generated sufficiently secure for the cryptographic mode of operation? Is an insecure mode of operation such as ECB in use? Is encryption used when authenticated encryption is more appropriate?
- Are passwords being used as cryptographic keys in the absence of a password based key derivation function?
- Is randomness used that was not designed to meet cryptographic requirements? Even if the correct function is chosen, does it need to be seeded by the developer, and if not, has the developer over-written the strong seeding functionality built into it with a seed that lacks sufficient entropy/unpredictability?
- Are deprecated hash functions such as MD5 or SHA1 in use, or are non-cryptographic hash functions used when cryptographic hash functions are needed?
- Are cryptographic error messages or side channel information exploitable, for example in the form of padding oracle attacks?
- Can the cryptographic algorithm be downgraded or bypassed?

See references ASVS: Cryptography (V11), Secure Communication (V12) and Data Protection (V14).

How to prevent.

Do the following, at a minimum, and consult the references:

- Classify and label data processed, stored, or transmitted by an application. Identify which data is sensitive according to privacy laws, regulatory requirements, or business needs.
- Store your most sensitive keys in a hardware or cloud-based HSM.
- Use well-trusted implementations of cryptographic algorithms whenever possible.
- Don't store sensitive data unnecessarily. Discard it as soon as possible or use PCI DSS compliant tokenization or even truncation. Data that is not retained cannot be stolen.
- Make sure to encrypt all sensitive data at rest.
- Ensure up-to-date and strong standard algorithms, protocols, and keys are in place; use proper key management.

- Encrypt all data in transit with protocols $\geq$ TLS 1.2 only, with forward secrecy (FS) ciphers, drop support for cipher block chaining (CBC) ciphers, support quantum key change algorithms. For HTTPS enforce encryption using HTTP Strict Transport Security (HSTS). Check everything with a tool.
- Disable caching for responses that contain sensitive data. This includes caching in your CDN, web server, and any application caching (eg: Redis).
- Apply required security controls as per the data classification.
- Do not use unencrypted protocols such as FTP, and STARTTLS. Avoid using SMTP for transmitting confidential data.
- Store passwords using strong adaptive and salted hashing functions with a work factor (delay factor), such as Argon2, yescrypt, scrypt or PBKDF2-HMAC-SHA-512. For legacy systems using bcrypt, get more advice at OWASP Cheat Sheet: Password Storage
- Initialization vectors must be chosen appropriate for the mode of operation. This could mean using a CSPRNG (cryptographically secure pseudo random number generator). For modes that require a nonce, the initialization vector (IV) does not need a CSPRNG. In all cases, the IV should never be used twice for a fixed key.
- Always use authenticated encryption instead of just encryption.
- Keys should be generated cryptographically randomly and stored in memory as byte arrays. If a password is used, then it must be converted to a key via an appropriate password base key derivation function.
- Ensure that cryptographic randomness is used where appropriate and that it has not been seeded in a predictable way or with low entropy. Most modern APIs do not require the developer to seed the CSPRNG to be secure.
- Avoid deprecated cryptographic functions, block building methods and padding schemes, such as MD5, SHA1, Cipher Block Chaining Mode (CBC), PKCS number 1 v1.5.
- Ensure settings and configurations meet security requirements by having them reviewed by security specialists, tools designed for this purpose, or both.
- You need to prepare now for post quantum cryptography (PQC), see reference (ENISA) so that high risk systems are safe no later than the end of 2030.

Example attack scenarios.

Scenario #1: A site doesn't use or enforce TLS for all pages or supports weak encryption. An attacker monitors network traffic (e.g., at an insecure wireless network), downgrades connections from HTTPS to HTTP, intercepts requests, and steals the user's session cookie. The attacker then replays this cookie and hijacks the user's (authenticated) session, accessing or modifying the user's private data. Instead of the above they could alter all transported data, e.g., the recipient of a money transfer.

Scenario #2: The password database uses unsalted or simple hashes to store everyone's passwords. A file upload flaw allows an attacker to retrieve the password database. All the unsalted hashes can be exposed with a rainbow table of pre-calculated hashes. Hashes generated by simple or fast hash functions may be cracked by GPUs, even if they were salted.

References.

- OWASP Proactive Controls: C2: Use Cryptography to Protect Data
- OWASP Application Security Verification Standard (ASVS): V11, 12, 14
- OWASP Cheat Sheet: Transport Layer Protection
- OWASP Cheat Sheet: User Privacy Protection
- OWASP Cheat Sheet: Password Storage
- OWASP Cheat Sheet: Cryptographic Storage
- OWASP Cheat Sheet: HSTS
- OWASP Testing Guide: Testing for weak cryptography
- ENISA: A Coordinated Implementation Roadmap for the Transition to Post-Quantum Cryptography
- NIST Releases First 3 Finalized Post-Quantum Encryption Standards

List of Mapped CWEs

- CWE-261 Weak Encoding for Password
- CWE-296 Improper Following of a Certificate's Chain of Trust
- CWE-319 Cleartext Transmission of Sensitive Information
- CWE-320 Key Management Errors (Prohibited)
- CWE-321 Use of Hard-coded Cryptographic Key
- CWE-322 Key Exchange without Entity Authentication
- CWE-323 Reusing a Nonce, Key Pair in Encryption
- CWE-324 Use of a Key Past its Expiration Date
- CWE-325 Missing Required Cryptographic Step
- CWE-326 Inadequate Encryption Strength
- CWE-327 Use of a Broken or Risky Cryptographic Algorithm
- CWE-328 Reversible One-Way Hash
- CWE-329 Not Using a Random IV with CBC Mode
- CWE-330 Use of Insufficiently Random Values
- CWE-331 Insufficient Entropy
- CWE-332 Insufficient Entropy in PRNG

- CWE-334 Small Space of Random Values
- CWE-335 Incorrect Usage of Seeds in Pseudo-Random Number Generator(PRNG)
- CWE-336 Same Seed in Pseudo-Random Number Generator (PRNG)
- CWE-337 Predictable Seed in Pseudo-Random Number Generator (PRNG)
- CWE-338 Use of Cryptographically Weak Pseudo-Random Number Generator(PRNG)
- CWE-340 Generation of Predictable Numbers or Identifiers
- CWE-342 Predictable Exact Value from Previous Values
- CWE-347 Improper Verification of Cryptographic Signature
- CWE-523 Unprotected Transport of Credentials
- CWE-757 Selection of Less-Secure Algorithm During Negotiation('Algorithm Downgrade')
- CWE-759 Use of a One-Way Hash without a Salt
- CWE-760 Use of a One-Way Hash with a Predictable Salt
- CWE-780 Use of RSA Algorithm without OAEP
- CWE-916 Use of Password Hash With Insufficient Computational Effort
- CWE-1240 Use of a Cryptographic Primitive with a Risky Implementation
- CWE-1241 Use of Predictable Algorithm in Random Number Generator

A05:2025 Injection

Background.

Injection falls two spots from #3 to #5 in the ranking, maintaining its position relative to A04:2025-Cryptographic Failures and A06:2025-Insecure Design. Injection is one of the most tested categories with 100% of applications tested for some form of injection. It had the greatest number of CVEs for any category, with 37 CWEs in this category. Injection includes Cross-site Scripting (high frequency/low impact) with more than 30k CVEs and SQL Injection (low frequency/high impact) with more than 14k CVEs. The massive number of reported CVEs for CWE-79 Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') brings down the average weighted impact of this category.

Score table.

CWEs Mapped
Max Incidence Rate
Avg Incidence Rate
Max Coverage
Avg Coverage
Avg Weighted Exploit
Avg Weighted Impact
Total Occurrences
Total CVEs

```
</td>
</tr>
<tr>
<td>37
</td>
<td>13.77%
</td>
<td>3.08%
</td>
<td>100.00%
</td>
<td>42.93%
</td>
<td>7.15
</td>
<td>4.32
</td>
<td>1,404,249
</td>
<td>62,445
</td>
</tr>
</table>
```

Description.

An injection vulnerability is an application flaw that allows untrusted user input to be sent to an interpreter (e.g. a browser, database, the command line) and causes the interpreter to execute parts of that input as commands.

An application is vulnerable to attack when:

- User-supplied data is not validated, filtered, or sanitized by the application.
- Dynamic queries or non-parameterized calls without context-aware escaping are used directly in the interpreter.

- Unsanitized data is used within object-relational mapping (ORM) search parameters to extract additional, sensitive records.
- Potentially hostile data is directly used or concatenated. The SQL or command contains the structure and malicious data in dynamic queries, commands, or stored procedures.

Some of the more common injections are SQL, NoSQL, OS command, Object Relational Mapping (ORM), LDAP, and Expression Language (EL) or Object Graph Navigation Library (OGNL) injection. The concept is identical among all interpreters. Detection is best achieved by a combination of source code review along with automated testing (including fuzzing) of all parameters, headers, URL, cookies, JSON, SOAP, and XML data inputs. The addition of static (SAST), dynamic (DAST), and interactive (IAST) application security testing tools into the CI/CD pipeline can also be helpful to identify injection flaws before production deployment.

A related class of injection vulnerabilities has become common in LLMs. These are discussed separately in the OWASP LLM Top 10, specifically LLM01:2025 Prompt Injection.

How to prevent.

The best means to prevent injection requires keeping data separate from commands and queries:

- The preferred option is to use a safe API, which avoids using the interpreter entirely, provides a parameterized interface, or migrates to Object Relational Mapping Tools (ORMs).

Note: Even when parameterized, stored procedures can still introduce SQL injection if PL/SQL or T-SQL concatenates queries and data or executes hostile data with EXECUTE IMMEDIATE or exec().

When it is not possible to separate the data from commands, you can reduce threats using the following techniques.

- Use positive server-side input validation. This is not a complete defense as many applications require special characters, such as text areas or APIs for mobile applications.
- For any residual dynamic queries, escape special characters using the specific escape syntax for that interpreter.

Note: SQL structures such as table names, column names, and so on cannot be escaped, and thus user-supplied structure names are dangerous. This is a common issue in report-writing software.

Warning these techniques involve parsing and escaping complex strings, making them error-prone and not robust in the face of minor changes to the underlying system.

Example attack scenarios.

Scenario #1: An application uses untrusted data in the construction of the following vulnerable SQL call:

```
String query = "SELECT * FROM accounts WHERE custID='" + request.getParameter("id") + "'";
```


An attacker modifies the 'id' parameter value in their browser to send: ' OR '1'='1. For example:

```
http://example.com/app/accountView?id=' OR '1'='1
```

This changes the meaning of the query to return all records from the accounts table. More dangerous attacks could modify or delete data or even invoke stored procedures.

Scenario #2: An application's blind trust in frameworks may result in queries that are still vulnerable. For example, Hibernate Query Language (HQL):

```
Query HQLQuery = session.createQuery("FROM accounts WHERE custID='" + request.getParameter("id") + "'");
```

An attacker supplies: ' OR custID IS NOT NULL OR custID='. This bypasses the filter and returns all accounts. While HQL has fewer dangerous functions than raw SQL, it still allows unauthorized data access when user input is concatenated into queries.

Scenario #3: An application passes user input directly to an OS command:

```
String cmd = "nslookup " + request.getParameter("domain");  
Runtime.getRuntime().exec(cmd);
```

An attacker supplies example.com; cat /etc/passwd to execute arbitrary commands on the server.

References.

- OWASP Proactive Controls: Secure Database Access
- OWASP ASVS: V5 Input Validation and Encoding
- OWASP Testing Guide: SQL Injection, Command Injection, and ORM Injection
- OWASP Cheat Sheet: Injection Prevention
- OWASP Cheat Sheet: SQL Injection Prevention
- OWASP Cheat Sheet: Injection Prevention in Java
- OWASP Cheat Sheet: Query Parameterization
- OWASP Automated Threats to Web Applications – OAT-014
- PortSwigger: Server-side template injection
- Awesome Fuzzing: a list of fuzzing resources

List of Mapped CWEs

- CWE-20 Improper Input Validation
- CWE-74 Improper Neutralization of Special Elements in Output Used by a Downstream Component ('Injection')

- CWE-76 Improper Neutralization of Equivalent Special Elements
- CWE-77 Improper Neutralization of Special Elements used in a Command ('Command Injection')
- CWE-78 Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')
- CWE-79 Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
- CWE-80 Improper Neutralization of Script-Related HTML Tags in a Web Page (Basic XSS)
- CWE-83 Improper Neutralization of Script in Attributes in a Web Page
- CWE-86 Improper Neutralization of Invalid Characters in Identifiers in Web Pages
- CWE-88 Improper Neutralization of Argument Delimiters in a Command ('Argument Injection')
- CWE-89 Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
- CWE-90 Improper Neutralization of Special Elements used in an LDAP Query ('LDAP Injection')
- CWE-91 XML Injection (aka Blind XPath Injection)
- CWE-93 Improper Neutralization of CRLF Sequences ('CRLF Injection')
- CWE-94 Improper Control of Generation of Code ('Code Injection')
- CWE-95 Improper Neutralization of Directives in Dynamically Evaluated Code ('Eval Injection')
- CWE-96 Improper Neutralization of Directives in Statically Saved Code ('Static Code Injection')
- CWE-97 Improper Neutralization of Server-Side Includes (SSI) Within a Web Page
- CWE-98 Improper Control of Filename for Include/Require Statement in PHP Program ('PHP Remote File Inclusion')
- CWE-99 Improper Control of Resource Identifiers ('Resource Injection')
- CWE-103 Struts: Incomplete validate() Method Definition
- CWE-104 Struts: Form Bean Does Not Extend Validation Class
- CWE-112 Missing XML Validation
- CWE-113 Improper Neutralization of CRLF Sequences in HTTP Headers ('HTTP Response Splitting')
- CWE-114 Process Control
- CWE-115 Misinterpretation of Output
- CWE-116 Improper Encoding or Escaping of Output
- CWE-129 Improper Validation of Array Index
- CWE-159 Improper Handling of Invalid Use of Special Elements

- CWE-470 Use of Externally-Controlled Input to Select Classes or Code ('Unsafe Reflection')
- CWE-493 Critical Public Variable Without Final Modifier
- CWE-500 Public Static Field Not Marked Final
- CWE-564 SQL Injection: Hibernate
- CWE-610 Externally Controlled Reference to a Resource in Another Sphere
- CWE-643 Improper Neutralization of Data within XPath Expressions ('XPath Injection')
- CWE-644 Improper Neutralization of HTTP Headers for Scripting Syntax
- CWE-917 Improper Neutralization of Special Elements used in an Expression Language Statement ('Expression Language Injection')

A06:2025 Insecure Design

Background.

Insecure Design slides two spots from #4 to #6 in the ranking as **A02:2025-Security Misconfiguration** and **A03:2025-Software Supply Chain Failures** leapfrog it. This category was introduced in 2021, and we have seen noticeable improvements in the industry related to threat modeling and a greater emphasis on secure design. This category focuses on risks related to design and architectural flaws, with a call for more use of threat modeling, secure design patterns, and reference architectures. This includes flaws in the business logic of an application, e.g. the lack of defining unwanted or unexpected state changes inside an application. As a community, we need to move beyond "shift-left" in the coding space, to pre-code activities such as requirements writing and application design, that are critical for the principles of Secure by Design (e.g. see **Establish a Modern AppSec Program: Planning and Design Phase**). Notable Common Weakness Enumerations (CWEs) include *CWE-256: Unprotected Storage of Credentials*, *CWE-269 Improper Privilege Management*, *CWE-434 Unrestricted Upload of File with Dangerous Type*, *CWE-501: Trust Boundary Violation*, and *CWE-522: Insufficiently Protected Credentials*.

Score table.

CWEs Mapped
Max Incidence Rate
Avg Incidence Rate
Max Coverage
Avg Coverage
Avg Weighted Exploit
Avg Weighted Impact

Total Occurrences
Total CVEs
39
22.18%
1.86%
88.76%
35.18%
6.96
4.05
729,882
7,647

Description.

Insecure design is a broad category representing different weaknesses, expressed as “missing or ineffective control design.” Insecure design is not the source for all other Top Ten risk categories. Note that there is a difference between insecure design and insecure implementation. We differentiate

between design flaws and implementation defects for a reason, they have different root causes, take place at different times in the development process, and have different remediations. A secure design can still have implementation defects leading to vulnerabilities that may be exploited. An insecure design cannot be fixed by a perfect implementation as needed security controls were never created to defend against specific attacks. One of the factors that contributes to insecure design is the lack of business risk profiling inherent in the software or system being developed, and thus the failure to determine what level of security design is required.

Three key parts of having a secure design are:

- Gathering Requirements and Resource Management
- Creating a Secure Design
- Having a Secure Development Lifecycle

Requirements and Resource Management

Collect and negotiate the business requirements for an application with the business, including the protection requirements concerning confidentiality, integrity, availability, and authenticity of all data assets and the expected business logic. Take into account how exposed your application will be and if you need segregation of tenants (beyond those needed for access control). Compile the technical requirements, including functional and non-functional security requirements. Plan and negotiate the budget covering all design, build, testing, and operation, including security activities.

Secure Design

Secure design is a culture and methodology that constantly evaluates threats and ensures that code is robustly designed and tested to prevent known attack methods. Threat modeling should be integrated into refinement sessions (or similar activities); look for changes in data flows and access control or other security controls. In the user story development, determine the correct flow and failure states, ensure they are well understood and agreed upon by the responsible and impacted parties. Analyze assumptions and conditions for expected and failure flows to ensure they remain accurate and desirable. Determine how to validate the assumptions and enforce conditions needed for proper behaviors. Ensure the results are documented in the user story. Learn from mistakes and offer positive incentives to promote improvements. Secure design is neither an add-on nor a tool that you can add to software.

Secure Development Lifecycle

Secure software requires a secure development lifecycle, a secure design pattern, a paved road methodology, a secure component library, appropriate tooling, threat modeling, and incident post-mortems that are used to improve the process. Reach out to your security specialists at the beginning of a software project, throughout the project, and for ongoing software maintenance. Consider leveraging the OWASP Software Assurance Maturity Model (SAMM) to help structure your secure software development efforts.

Often self-responsibility of developers is underappreciated. Foster a culture of awareness, responsibility and proactive risk mitigation. Regular exchanges about security (e.g. during threat modeling sessions) can generate a mindset for including security in all important design decisions.

How to prevent.

- Establish and use a secure development lifecycle with AppSec professionals to help evaluate and design security and privacy-related controls
- Establish and use a library of secure design patterns or paved-road components
- Use threat modeling for critical parts of the application such as authentication, access control, business logic, and key flows
- User threat modeling as an educational tool to generate a security mindset
- Integrate security language and controls into user stories
- Integrate plausibility checks at each tier of your application (from frontend to backend)
- Write unit and integration tests to validate that all critical flows are resistant to the threat model. Compile use-cases *and* misuse-cases for each tier of your application.
- Segregate tier layers on the system and network layers, depending on the exposure and protection needs
- Segregate tenants robustly by design throughout all tiers

Example attack scenarios.

Scenario #1: A credential recovery workflow might include “questions and answers,” which is prohibited by NIST 800-63b, the OWASP ASVS, and the OWASP Top 10. Questions and answers cannot be trusted as evidence of identity, as more than one person can know the answers. Such functionality should be removed and replaced with a more secure design.

Scenario #2: A cinema chain allows group booking discounts and has a maximum of fifteen attendees before requiring a deposit. Attackers could threat model this flow and test if they can find an attack vector in the business logic of the application, e.g. booking six hundred seats and all cinemas at once in a few requests, causing a massive loss of income.

Scenario #3: A retail chain's e-commerce website does not have protection against bots run by scalpers buying high-end video cards to resell on auction websites. This creates terrible publicity for the video card makers and retail chain owners, and enduring bad blood with enthusiasts who cannot obtain these cards at any price. Careful anti-bot design and domain logic rules, such as purchases made within a few seconds of availability, might identify inauthentic purchases and reject such transactions.

References.

- OWASP Cheat Sheet: Secure Design Principles

- OWASP SAMM: Design | Secure Architecture
- OWASP SAMM: Design | Threat Assessment
- NIST – Guidelines on Minimum Standards for Developer Verification of Software
- The Threat Modeling Manifesto
- Awesome Threat Modeling

List of Mapped CWEs

- CWE-73 External Control of File Name or Path
- CWE-183 Permissive List of Allowed Inputs
- CWE-256 Unprotected Storage of Credentials
- CWE-266 Incorrect Privilege Assignment
- CWE-269 Improper Privilege Management
- CWE-286 Incorrect User Management
- CWE-311 Missing Encryption of Sensitive Data
- CWE-312 Cleartext Storage of Sensitive Information
- CWE-313 Cleartext Storage in a File or on Disk
- CWE-316 Cleartext Storage of Sensitive Information in Memory
- CWE-362 Concurrent Execution using Shared Resource with Improper Synchronization ('Race Condition')
- CWE-382 J2EE Bad Practices: Use of System.exit()
- CWE-419 Unprotected Primary Channel
- CWE-434 Unrestricted Upload of File with Dangerous Type
- CWE-436 Interpretation Conflict
- CWE-444 Inconsistent Interpretation of HTTP Requests ('HTTP Request Smuggling')
- CWE-451 User Interface (UI) Misrepresentation of Critical Information
- CWE-454 External Initialization of Trusted Variables or Data Stores
- CWE-472 External Control of Assumed-Immutable Web Parameter
- CWE-501 Trust Boundary Violation
- CWE-522 Insufficiently Protected Credentials
- CWE-525 Use of Web Browser Cache Containing Sensitive Information

- CWE-539 Use of Persistent Cookies Containing Sensitive Information
- CWE-598 Use of GET Request Method With Sensitive Query Strings
- CWE-602 Client-Side Enforcement of Server-Side Security
- CWE-628 Function Call with Incorrectly Specified Arguments
- CWE-642 External Control of Critical State Data
- CWE-646 Reliance on File Name or Extension of Externally-Supplied File
- CWE-653 Insufficient Compartmentalization
- CWE-656 Reliance on Security Through Obscurity
- CWE-657 Violation of Secure Design Principles
- CWE-676 Use of Potentially Dangerous Function
- CWE-693 Protection Mechanism Failure
- CWE-799 Improper Control of Interaction Frequency
- CWE-807 Reliance on Untrusted Inputs in a Security Decision
- CWE-841 Improper Enforcement of Behavioral Workflow
- CWE-1021 Improper Restriction of Rendered UI Layers or Frames
- CWE-1022 Use of Web Link to Untrusted Target with window.opener Access
- CWE-1125 Excessive Attack Surface

A07:2025 Authentication Failures

Background.

Authentication Failures maintains its position at #7 with a slight name change to more accurately reflect the 36 CWEs in this category. Despite benefits from standardized frameworks, this category has kept its #7 rank from 2021. Notable CWEs included are *CWE-259 Use of Hard-coded Password*, *CWE-297: Improper Validation of Certificate with Host Mismatch*, *CWE-287: Improper Authentication*, *CWE-384: Session Fixation*, and *CWE-798 Use of Hard-coded Credentials*.

Score table.

CWEs Mapped
Max Incidence Rate
Avg Incidence Rate
Max Coverage
Avg Coverage
Avg Weighted Exploit
Avg Weighted Impact
Total Occurrences
Total CVEs

```
<tr>
<td>36
</td>
<td>15.80%
</td>
<td>2.92%
</td>
<td>100.00%
</td>
<td>37.14%
</td>
<td>7.69
</td>
<td>4.44
</td>
<td>1,120,673
</td>
<td>7,147
</td>
</tr>
</table>
```

Description.

When an attacker is able to trick a system into recognizing an invalid or incorrect user as legitimate, this vulnerability is present. There may be authentication weaknesses if the application:

- Permits automated attacks such as credential stuffing, where the attacker has a breached list of valid usernames and passwords. More recently this type of attack has been expanded to include hybrid password attacks credential stuffing (also known as password spray attacks), where the attacker uses variations or increments of spilled credentials to gain access, for instance trying Password1!, Password2!, Password3! and so on.
- Permits brute force or other automated, scripted attacks that are not quickly blocked.
- Permits default, weak, or well-known passwords, such as "Password1" or "admin" username with an "admin" password.

- Allows users to create new accounts with already known-breached credentials.
- Allows use of weak or ineffective credential recovery and forgot-password processes, such as "knowledge-based answers," which cannot be made safe.
- Uses plain text, encrypted, or weakly hashed passwords data stores (see A04:2025-Cryptographic Failures).
- Has missing or ineffective multi-factor authentication.
- Allows use of weak or ineffective fallbacks if multi-factor authentication is not available.
- Exposes session identifier in the URL, a hidden field, or another insecure location that is accessible to the client.
- Reuses the same session identifier after successful login.
- Does not correctly invalidate user sessions or authentication tokens (mainly single sign-on (SSO) tokens) during logout or a period of inactivity.
- Does not correctly assert the scope and intended audience of the provided credentials.

How to prevent.

- Where possible, implement and enforce use of multi-factor authentication to prevent automated credential stuffing, brute force, and stolen credential reuse attacks.
- Where possible, encourage and enable the use of password managers, to help users make better choices.
- Do not ship or deploy with any default credentials, particularly for admin users.
- Implement weak password checks, such as testing new or changed passwords against the top 10,000 worst passwords list.
- During new account creation and password changes validate against lists of known breached credentials (eg: using haveibeenpwned.com).
- Align password length, complexity, and rotation policies with National Institute of Standards and Technology (NIST) 800-63b's guidelines in section 5.1.1 for Memorized Secrets or other modern, evidence-based password policies.
- Do not force human beings to rotate passwords unless you suspect breach. If you suspect breach, force password resets immediately.
- Ensure registration, credential recovery, and API pathways are hardened against account enumeration attacks by using the same messages for all outcomes ("Invalid username or password.").
- Limit or increasingly delay failed login attempts but be careful not to create a denial of service scenario. Log all failures and alert administrators when credential stuffing, brute force, or other

attacks are detected or suspected.

- Use a server-side, secure, built-in session manager that generates a new random session ID with high entropy after login. Session identifiers should not be in the URL, be securely stored in a secure cookie, and invalidated after logout, idle, and absolute timeouts.
- Ideally, use a premade, well-trusted system to handle authentication, identity, and session management. Transfer this risk whenever possible by buying and utilizing a hardened and well tested system.
- Verify the intended use of provided credentials, e.g. for JWTs validate aud, iss claims and scopes

Example attack scenarios.

Scenario #1: Credential stuffing, the use of lists of known username and password combinations, is now a very common attack. More recently attackers have been found to 'increment' or otherwise adjust passwords, based on common human behavior. For instance, changing 'Winter2025' to 'Winter2026', or 'ILoveMyDog6' to 'ILoveMyDog7' or 'ILoveMyDog5'. This adjusting of password attempts is called a hybrid credential stuffing attack or a password spray attack, and they can be even more effective than the traditional version. If an application does not implement defences against automated threats (brute force, scripts, or bots) or credential stuffing, the application can be used as a password oracle to determine if the credentials are valid and gain unauthorized access.

Scenario #2: Most successful authentication attacks occur due to the continued use of passwords as the sole authentication factor. Once considered best practices, password rotation and complexity requirements encourage users to both reuse passwords and use weak passwords. Organizations are recommended to stop these practices per NIST 800-63 and to enforce use of multi-factor authentication on all important systems.

Scenario #3: Application session timeouts aren't implemented correctly. A user uses a public computer to access an application and instead of selecting "logout," the user simply closes the browser tab and walks away. Another Example for this is, if a Single Sign on (SSO) session can not be closed by a Single Logout (SLO). That is, a single login logs you into, for example, your mail reader, your document system, and your chat system. But logging out happens only to the current system. If an attacker uses the same browser after the victim thinks they have successfully logged out, but with the user still authenticated to some of the applications, then can access the victim's account. The same issue can happen in offices and enterprises when a sensitive application has not been properly exited and a colleague has (temporary) access to the unlocked computer.

References.

- OWASP Authentication Cheat Sheet
- OWASP Secure Coding Practices

List of Mapped CWEs

- CWE-258 Empty Password in Configuration File
- CWE-259 Use of Hard-coded Password
- CWE-287 Improper Authentication
- CWE-288 Authentication Bypass Using an Alternate Path or Channel
- CWE-289 Authentication Bypass by Alternate Name
- CWE-290 Authentication Bypass by Spoofing
- CWE-291 Reliance on IP Address for Authentication
- CWE-293 Using Referer Field for Authentication
- CWE-294 Authentication Bypass by Capture-replay
- CWE-295 Improper Certificate Validation
- CWE-297 Improper Validation of Certificate with Host Mismatch
- CWE-298 Improper Validation of Certificate with Host Mismatch
- CWE-299 Improper Validation of Certificate with Host Mismatch
- CWE-300 Channel Accessible by Non-Endpoint
- CWE-302 Authentication Bypass by Assumed-Immutable Data
- CWE-303 Incorrect Implementation of Authentication Algorithm
- CWE-304 Missing Critical Step in Authentication
- CWE-305 Authentication Bypass by Primary Weakness
- CWE-306 Missing Authentication for Critical Function
- CWE-307 Improper Restriction of Excessive Authentication Attempts
- CWE-308 Use of Single-factor Authentication
- CWE-309 Use of Password System for Primary Authentication
- CWE-346 Origin Validation Error
- CWE-350 Reliance on Reverse DNS Resolution for a Security-Critical Action
- CWE-384 Session Fixation
- CWE-521 Weak Password Requirements
- CWE-613 Insufficient Session Expiration

- CWE-620 Unverified Password Change
- CWE-640 Weak Password Recovery Mechanism for Forgotten Password
- CWE-798 Use of Hard-coded Credentials
- CWE-940 Improper Verification of Source of a Communication Channel
- CWE-941 Incorrectly Specified Destination in a Communication Channel
- CWE-1390 Weak Authentication
- CWE-1391 Use of Weak Credentials
- CWE-1392 Use of Default Credentials
- CWE-1393 Use of Default Password

A08:2025 Software or Data Integrity Failures

Background.

Software or Data Integrity Failures continues at #8, with a slight, clarifying name change from "Software *and* Data Integrity Failures". This category is focused on the failure to maintain trust boundaries and verify the integrity of software, code, and data artifacts at a lower level than Software Supply Chain Failures. This category focuses on making assumptions related to software updates and critical data, without verifying integrity. Notable Common Weakness Enumerations (CWEs) include *CWE-829: Inclusion of Functionality from Untrusted Control Sphere*, *CWE-915: Improperly Controlled Modification of Dynamically-Determined Object Attributes*, and *CWE-502: Deserialization of Untrusted Data*.

Score table.

CWEs Mapped
Max Incidence Rate
Avg Incidence Rate
Max Coverage
Avg Coverage
Avg Weighted Exploit
Avg Weighted Impact
Total Occurrences
Total CVEs


```
</td>
</tr>
<tr>
<td>14
</td>
<td>8.98%
</td>
<td>2.75%
</td>
<td>78.52%
</td>
<td>45.49%
</td>
<td>7.11
</td>
<td>4.79
</td>
<td>501,327
</td>
<td>3,331
</td>
</tr>
</table>
```

Description.

Software and data integrity failures relate to code and infrastructure that does not protect against invalid or untrusted code or data being treated as trusted and valid. An example of this is where an application relies upon plugins, libraries, or modules from untrusted sources, repositories, and content delivery networks (CDNs). An insecure CI/CD pipeline without consuming and providing software integrity checks can introduce the potential for unauthorized access, insecure or malicious code, or system compromise. Another example of this is a CI/CD that pulls code or artifacts from untrusted places and/or doesn't verify them before use (by checking the signature or similar mechanism). Lastly, many applications now include auto-update functionality, where updates are downloaded without sufficient integrity verification and applied to the previously trusted application. Attackers could potentially upload

their own updates to be distributed and run on all installations. Another example is where objects or data are encoded or serialized into a structure that an attacker can see and modify is vulnerable to insecure deserialization.

How to prevent.

- Use digital signatures or similar mechanisms to verify the software or data is from the expected source and has not been altered.
- Ensure libraries and dependencies, such as npm or Maven, are only consuming trusted repositories. If you have a higher risk profile, consider hosting an internal known-good repository that's vetted.
- Ensure that there is a review process for code and configuration changes to minimize the chance that malicious code or configuration could be introduced into your software pipeline.
- Ensure that your CI/CD pipeline has proper segregation, configuration, and access control to ensure the integrity of the code flowing through the build and deploy processes.
- Ensure that unsigned or unencrypted serialized data is not received from untrusted clients and subsequently used without some form of integrity check or digital signature to detect tampering or replay of the serialized data.

Example attack scenarios.

Scenario #1 Inclusion of Web Functionality from an Untrusted Source: A company uses an external service provider to provide support functionality. For convenience, it has a DNS mapping for `myCompany.SupportProvider.com` to `support.myCompany.com`. This means that all cookies, including authentication cookies, set on the `myCompany.com` domain will now be sent to the support provider. Anyone with access to the support provider's infrastructure can steal the cookies of all of your users that have visited `support.myCompany.com` and perform a session hijacking attack.

Scenario #2 Update without signing: Many home routers, set-top boxes, device firmware, and others do not verify updates via signed firmware. Unsigned firmware is a growing target for attackers and is expected to only get worse. This is a major concern as many times there is no mechanism to remediate other than to fix in a future version and wait for previous versions to age out.

Scenario #3 Use of Package from an Untrusted Source: A developer has trouble finding the updated version of a package they are looking for, so they download it not from the regular, trusted package manager, but from a website online. The package is not signed, and thus there is no opportunity to ensure integrity. The package includes malicious code.

Scenario #4 Insecure Deserialization: A React application calls a set of Spring Boot microservices. Being functional programmers, they tried to ensure that their code is immutable. The solution they came up with is serializing the user state and passing it back and forth with each request. An attacker notices the "rOO" Java object signature (in base64) and uses the Java Deserialization Scanner to gain remote code execution on the application server.

References.

- OWASP Cheat Sheet: Software Supply Chain Security
- OWASP Cheat Sheet: Infrastructure as Code
- OWASP Cheat Sheet: Deserialization
- SAFECode Software Integrity Controls
- A 'Worst Nightmare' Cyberattack: The Untold Story Of The SolarWinds Hack
- CodeCov Bash Uploader Compromise
- Securing DevOps by Julien Vehent
- Insecure Deserialization by Tenendo

List of Mapped CWEs

- CWE-345 Insufficient Verification of Data Authenticity
- CWE-353 Missing Support for Integrity Check
- CWE-426 Untrusted Search Path
- CWE-427 Uncontrolled Search Path Element
- CWE-494 Download of Code Without Integrity Check
- CWE-502 Deserialization of Untrusted Data
- CWE-506 Embedded Malicious Code
- CWE-509 Replicating Malicious Code (Virus or Worm)
- CWE-565 Reliance on Cookies without Validation and Integrity Checking
- CWE-784 Reliance on Cookies without Validation and Integrity Checking in a Security Decision
- CWE-829 Inclusion of Functionality from Untrusted Control Sphere
- CWE-830 Inclusion of Web Functionality from an Untrusted Source
- CWE-915 Improperly Controlled Modification of Dynamically-Determined Object Attributes
- CWE-926 Improper Export of Android Application Components

A09:2025 Security Logging & Alerting Failures

Background.

Security Logging & Alerting Failures retains its position at #9. This category has a slight name change to emphasize the alerting function needed to induce action on relevant logging events. This category will always be underrepresented in the data, and for the third time voted into a position in the list from the community survey participants. This category is incredibly difficult to test for, and has minimal representation in the CVE/CVSS data (only 723 CVEs); but can be very impactful for visibility and incident alerting and forensics. This category includes issues with *properly handling output encoding to log files (CWE-117)*, *inserting sensitive data into log files (CWE-532)*, and *insufficient logging (CWE-778)*.

Score table.

CWEs Mapped
Max Incidence Rate
Avg Incidence Rate
Max Coverage
Avg Coverage
Avg Weighted Exploit
Avg Weighted Impact
Total Occurrences
Total CVEs

```
</td>
</tr>
<tr>
<td>5
</td>
<td>11.33%
</td>
<td>3.91%
</td>
<td>85.96%
</td>
<td>46.48%
</td>
<td>7.19
</td>
<td>2.65
</td>
<td>260,288
</td>
<td>723
</td>
</tr>
</table>
```

Description.

Without logging and monitoring, attacks and breaches cannot be detected, and without alerting it is very difficult to respond quickly and effectively during a security incident. Insufficient logging, continuous monitoring, detection, and alerting to initiate active responses occurs any time:

- Auditable events, such as logins, failed logins, and high-value transactions, are not logged or logged inconsistently (for instance, only logging successful logins, but not failed attempts).
- Warnings and errors generate no, inadequate, or unclear log messages.
- The integrity of logs is not properly protected from tampering.

- Logs of applications and APIs are not monitored for suspicious activity.
- Logs are only stored locally, and not properly backed up.
- Appropriate alerting thresholds and response escalation processes are not in place or effective. Alerts are not received or reviewed within a reasonable amount of time.
- Penetration testing and scans by dynamic application security testing (DAST) tools (such as Burp or ZAP) do not trigger alerts.
- The application cannot detect, escalate, or alert for active attacks in real-time or near real-time.
- You are vulnerable to sensitive information leakage by making logging and alerting events visible to a user or an attacker (see A01:2025-Broken Access Control), or by logging sensitive information that should not be logged (such as PII or PHI).
- You are vulnerable to injections or attacks on the logging or monitoring systems if log data is not correctly encoded.
- The application is missing or mishandling errors and other exceptional conditions, such that the system is unaware there was an error, and is therefore unable to log there was a problem.
- Adequate 'use cases' for issuing alerts are missing or outdated to recognize a special situation.
- Too many false positive alerts make it impossible to distinguish important alerts from unimportant ones, resulting in them being recognized too late or not at all (physical overload of the SOC team).
- Detected alerts cannot be processed correctly because the playbook for the use case is incomplete, out of date, or missing.

How to prevent.

Developers should implement some or all the following controls, depending on the risk of the application:

- Ensure all login, access control, and server-side input validation failures can be logged with sufficient user context to identify suspicious or malicious accounts and held for enough time to allow delayed forensic analysis.
- Ensure that every part of your app that contains a security control is logged, whether it succeeds or fails.
- Ensure that logs are generated in a format that log management solutions can easily consume.
- Ensure log data is encoded correctly to prevent injections or attacks on the logging or monitoring systems.
- Ensure all transactions have an audit trail with integrity controls to prevent tampering or deletion, such as append-only database tables or similar.
- Ensure all transactions that throw an error are rolled back and started over. Always fail closed.
- If your application or its users behave suspiciously, issue an alert. Create guidance for your developers on this topic so they can code against this or buy a system for this.
- DevSecOps and security teams should establish effective monitoring and alerting use cases including playbooks such that suspicious activities are detected and responded to quickly by the

Security Operations Center (SOC) team.

- Add 'honeytokens' as traps for attackers into your application e.g. into the database, data, as real and/or technical user identity. As they are not used in normal business, any access generates logging data that can be alerted with nearly no false positives.
- Behavior analysis and AI support could be optionally an additional technique to support low rates of false positives for alerts.
- Establish or adopt an incident response and recovery plan, such as National Institute of Standards and Technology (NIST) 800-61r2 or later. Teach your software developers what application attacks and incidents look like, so they can report them.

There are commercial and open-source application protection products such as the OWASP ModSecurity Core Rule Set, and open-source log correlation software, such as the Elasticsearch, Logstash, Kibana (ELK) stack, that feature custom dashboards and alerting that may help you combat these issues. There are also commercial observability tools that can help you respond to or block attacks in close to real-time.

Example attack scenarios.

Scenario #1: A children's health plan provider's website operator couldn't detect a breach due to a lack of monitoring and logging. An external party informed the health plan provider that an attacker had accessed and modified thousands of sensitive health records of more than 3.5 million children. A post-incident review found that the website developers had not addressed significant vulnerabilities. As there was no logging or monitoring of the system, the data breach could have been in progress since 2013, a period of more than seven years.

Scenario #2: A major Indian airline had a data breach involving more than ten years' worth of personal data of millions of passengers, including passport and credit card data. The data breach occurred at a third-party cloud hosting provider, who notified the airline of the breach after some time.

Scenario #3: A major European airline suffered a GDPR reportable breach. The breach was reportedly caused by payment application security vulnerabilities exploited by attackers, who harvested more than 400,000 customer payment records. The airline was fined 20 million pounds as a result by the privacy regulator.

References.

- OWASP Proactive Controls: C9: Implement Logging and Monitoring
- OWASP Application Security Verification Standard: V16 Security Logging and Error Handling
- OWASP Cheat Sheet: Application Logging Vocabulary
- OWASP Cheat Sheet: Logging
- Data Integrity: Recovering from Ransomware and Other Destructive Events

- Data Integrity: Identifying and Protecting Assets Against Ransomware and Other Destructive Events
- Data Integrity: Detecting and Responding to Ransomware and Other Destructive Events
- Real world example of such failures in Snowflake Breach

List of Mapped CWEs

- CWE-117 Improper Output Neutralization for Logs
- CWE-221 Information Loss or Omission
- CWE-223 Omission of Security-relevant Information
- CWE-532 Insertion of Sensitive Information into Log File
- CWE-778 Insufficient Logging

A10:2025 Mishandling of Exceptional Conditions

Background.

Mishandling of Exceptional Conditions is a new category for 2025. This category contains 24 CWEs and focuses on improper error handling, logical errors, failing open, and other related scenarios stemming from abnormal conditions and systems may encounter. This category has some CWEs that were previously associated with poor code quality. That was too general for us; in our opinion, this more specific category provides better guidance.

Notable CWEs included in this category: *CWE-209 Generation of Error Message Containing Sensitive Information*, *CWE-234 Failure to Handle Missing Parameter*, *CWE-274 Improper Handling of Insufficient Privileges*, *CWE-476 NULL Pointer Dereference*, and *CWE-636 Not Failing Securely ('Failing Open')*.

Score table.

CWEs Mapped
Max Incidence Rate
Avg Incidence Rate
Max Coverage
Avg Coverage
Avg Weighted Exploit
Avg Weighted Impact

<td>Total Occurrences

</td>

<td>Total CVEs

</td>

</tr>

<tr>

<td>24

</td>

<td>20.67%

</td>

<td>2.95%

</td>

<td>100.00%

</td>

<td>37.95%

</td>

<td>7.11

</td>

<td>3.81

</td>

<td>769,581

</td>

<td>3,416

</td>

</tr>

</table>

Description.

Mishandling exceptional conditions in software happens when programs fail to prevent, detect, and respond to unusual and unpredictable situations, which leads to crashes, unexpected behavior, and sometimes vulnerabilities. This can involve one or more of the following 3 failings; the application doesn't prevent an unusual situation from happening, it doesn't identify the situation as it is happening, and/or it responds poorly or not at all to the situation afterwards.

Exceptional conditions can be caused by missing, poor, or incomplete input validation, or late, high level error handling instead at the functions where they occur, or unexpected environmental states such as memory, privilege, or network issues, inconsistent exception handling, or exceptions that are not handled at all, allowing the system to fall into an unknown and unpredictable state. Any time an application is unsure of its next instruction, an exceptional condition has been mishandled. Hard-to-find errors and exceptions can threaten the security of the whole application for a long time.

Many different security vulnerabilities can happen when we mishandle exceptional conditions,

such as logic bugs, overflows, race conditions, fraudulent transactions, or issues with memory, state, resource, timing, authentication, and authorization. These types of vulnerabilities can negatively affect the confidentiality, availability, and/or integrity of a system or its data. Attackers manipulate an application's flawed error handling to strike this vulnerability.

How to prevent.

In order to handle an exceptional condition properly we must plan for such situations (expect the worst). We must 'catch' every possible system error directly at the place where they occur and then handle it (which means do something meaningful to solve the problem and ensure we recover from the issue). As part of the handling, we should include throwing an error (to inform the user in an understandable way), logging of the event, as well as issuing an alert if we feel that is justified. We should also have a global exception handler in place in case there is ever something we have missed. Ideally, we would also have monitoring and/or observability tooling or functionality that watches for repeated errors or patterns that indicate an on-going attack, that could issue a response, defense, or blocking of some kind. This can help us block and respond to scripts and bots that focus on our error handling weaknesses.

Catching and handling exceptional conditions ensures that the underlying infrastructure of our programs are not left to deal with unpredictable situations. If you are part way through a transaction of any kind, it is extremely important that you roll back every part of the transaction and start again (also known as failing closed). Attempting to recover a transaction part way through is often where we create unrecoverable mistakes.

Whenever possible, add rate limiting, resource quotas, throttling, and other limits wherever possible, to prevent exceptional conditions in the first place. Nothing in information technology should be limitless, as this leads to a lack of application resilience, denial of service, successful brute force attacks, and extraordinary cloud bills. \

Consider whether identical repeated errors, above a certain rate, should only be outputted as statistics showing how often they have occurred and in what time frame. This information should be appended to the original message so as not to interfere with automated logging and monitoring, see A09:2025 Security Logging & Alerting Failures.

On top of this, we would want to include strict input validation (with sanitization or escaping for potentially hazardous characters that we must accept), and *centralized* error handling, logging, monitoring, and alerting, and a global exception handler. One application should not multiple functions

for handling exceptional conditions, it should be performed in one place, the same way each time. We should also create project security requirements for all the advice in this section, perform threat modelling and/or secure design review activities in the design phase of our projects, perform code review or static analysis, as well as execute stress, performance, and penetration testing of the final system.

If possible, your entire organization should handle exceptional conditions in the same way, as it makes it easier to review and audit code for errors in this important security control.

Example attack scenarios.

Scenario #1: Resource exhaustion via mishandling of exceptional conditions (Denial of Service) could be caused if the application catches exceptions when files are uploaded, but doesn't properly release resources after. Each new exception leaves resources locked or otherwise unavailable, until all resources are used up.

Scenario #2: Sensitive data exposure via improper handling or database errors that reveals the full system error to the user. The attacker continues to force errors in order to use the sensitive system information to create a better SQL injection attack. The sensitive data in the user error messages are reconnaissance.

Scenario #3: State corruption in financial transactions could be caused by an attacker interrupting a multi-step transaction via network disruptions. Imagine the transaction order was: debit user account, credit destination account, log transaction. If the system doesn't properly roll back the entire transaction (fail closed) when there is an error part way through, the attacker could potentially drain the user's account, or possibly a race condition that allows the attacker to send money to the destination multiple times.

References.

OWASP MASVS■RESILIENCE

- OWASP Cheat Sheet: Logging
- OWASP Cheat Sheet: Error Handling
- OWASP Application Security Verification Standard (ASVS): V16.5 Error Handling
- OWASP Testing Guide: 4.8.1 Testing for Error Handling
- Best practices for exceptions (Microsoft, .Net)
- Clean Code and the Art of Exception Handling (Toptal)
- General error handling rules (Google for Developers)
- Example of real-world mishandling of an exceptional condition

List of Mapped CWEs

- CWE-209 Generation of Error Message Containing Sensitive Information
- CWE-215 Insertion of Sensitive Information Into Debugging Code
- CWE-234 Failure to Handle Missing Parameter
- CWE-235 Improper Handling of Extra Parameters
- CWE-248 Uncaught Exception
- CWE-252 Unchecked Return Value
- CWE-274 Improper Handling of Insufficient Privileges
- CWE-280 Improper Handling of Insufficient Permissions or Privileges
- CWE-369 Divide By Zero
- CWE-390 Detection of Error Condition Without Action
- CWE-391 Unchecked Error Condition
- CWE-394 Unexpected Status Code or Return Value
- CWE-396 Declaration of Catch for Generic Exception
- CWE-397 Declaration of Throws for Generic Exception
- CWE-460 Improper Cleanup on Thrown Exception
- CWE-476 NULL Pointer Dereference
- CWE-478 Missing Default Case in Multiple Condition Expression
- CWE-484 Omitted Break Statement in Switch
- CWE-550 Server-generated Error Message Containing Sensitive Information
- CWE-636 Not Failing Securely ('Failing Open')
- CWE-703 Improper Check or Handling of Exceptional Conditions
- CWE-754 Improper Check for Unusual or Exceptional Conditions
- CWE-755 Improper Handling of Exceptional Conditions
- CWE-756 Missing Custom Error Page